

STATISTISCHES LERNEN 2

Aula 4

Probabilistic View &

Bayesian Regularization

Prof. Johannes Schwab, PhD

FH Kufstein Tirol

Visao Probabilistica, Bayesian Regularization & Bootstrapping

Esta aula revela o **fundamento probabilistico** de tudo que aprendemos ate agora. O MSE nao e arbitrario — emerge da hipotese de ruído Gaussiano. A regularizacao nao e um truque — e a incorporacao de conhecimento previo via **prior Bayesiano**. Depois da teoria, exploramos o **protocolo pratico** de escolha de hiperparametros, o diagnostico de Under vs. Over-regularizacao, e encerramos com **Bootstrapping** — a tecnica de resampling para quantificar incerteza e a base do Bagging (Random Forests).

#	Topico	Conceito Central
1	Probabilistic View	Likelihood, Posterior, MAP — por que MSE e regularizacao emergem naturalmente
2	Prior Choices	Gaussian Prior => L2 Ridge Laplace Prior => L1 Lasso
3	Decision Process	Quando usar qual regularizacao — 4 cenarios do slide
4	Hyperparameter Choice	Log-grid, Coarse-to-Fine, Multi-seed, Protocolo completo
5	Diagnosis	Under vs. Over-regularizacao via Learning Curves
6	Cross-Validation	Hold-Out, K-Fold, Stratified, LOO — comparacao quantitativa
7	Bootstrapping	Resampling com reposicao, Confidence Intervals, Bagging

1. Probabilistic View — Por que MSE e Regularizacao emergem do Bayes

Ate agora, treinar um modelo significava **minimizar um erro** (ex: MSE). A **Probabilistic View** pergunta algo diferente: dado que observamos os dados $D = \{(x_i, y_i)\}$, qual conjunto de parametros theta e mais **provavel**? Essa mudanca conecta dois conceitos:

- **Likelihood $p(D|\theta)$** : 'Se o modelo tivesse estes parametros, quao provaveis seriam os dados observados?'
- **Prior $p(\theta)$** : 'Antes de ver qualquer dado, qual configuracao de theta acredito ser mais provavel?'
- **Posterior $p(\theta|D)$** : 'Dado que vi estes dados, quao provavel e cada configuracao de theta?'
- **MAP (Maximum A Posteriori)**: escolher theta que maximiza o Posterior — e exatamente o que fazemos no treino com loss + regularizador.

Passo 1 — Likelihood com ruído Gaussiano:

$$p(D|\theta) = \prod_{i=1}^N \exp(-\|f_{\theta}(x_i) - y_i\|^2) \Rightarrow -\log p(D|\theta) = \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 \equiv \text{MSE}$$

-log Likelihood com ruído Gaussiano = MSE / SSE — nao e arbitrario!

Passo 2 — Teorema de Bayes:

$$p(\theta|D) \propto p(D|\theta) \cdot p(\theta)$$

Posterior proporcional a Likelihood x Prior

Passo 3 — Objetivo MAP (minimizar -log Posterior):

$$\hat{\theta}_{\text{MAP}} = \arg \min_{\theta} [-\log p(D|\theta) - \log p(\theta)] = \arg \min_{\theta} [\text{MSE} + \lambda \mathcal{R}(\theta)]$$

$-\log p(D|\theta) = \text{MSE}$ | $-\log p(\theta) = \text{Regularizacao}$ | λ conecta os dois

Este resultado é poderoso: o treinamento com **Loss + Regularizador** é **idêntico** a encontrar o estimador MAP. A regularização não é um truque ad-hoc — é a incorporação rigorosa de conhecimento a priori via Prior Bayesiano.

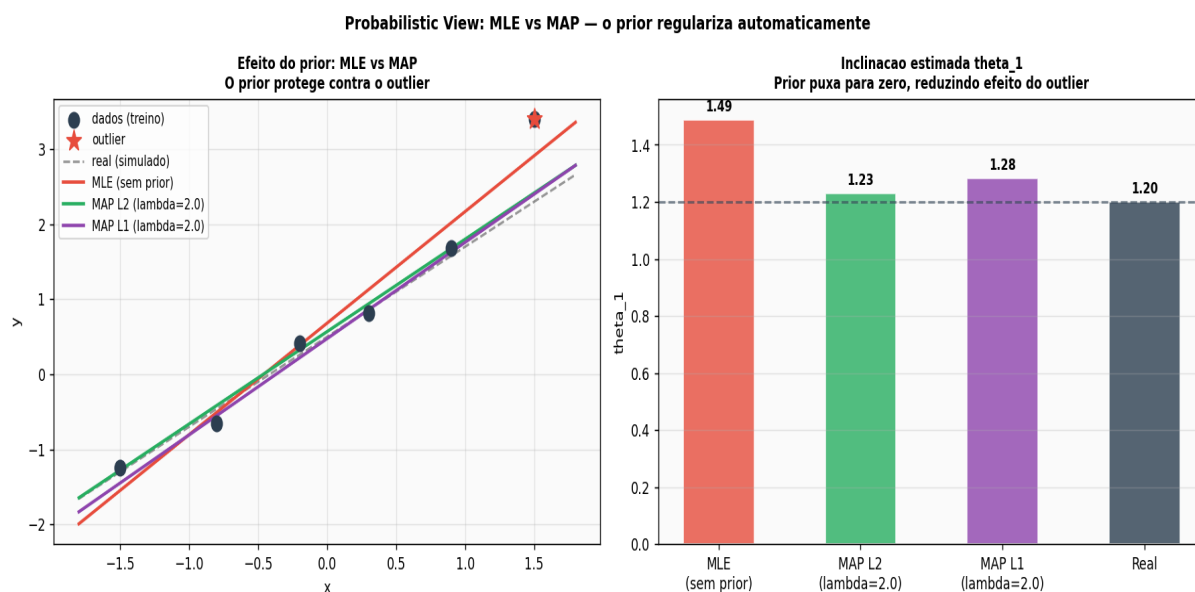


Figura 1 — Dataset com um outlier. MLE (vermelho) é muito influenciado pelo outlier e estima uma inclinação muito maior que a real. MAP com L2 (verde) e MAP com L1 (roxo) são 'puxados' pelo prior para valores menores, se aproximando do verdadeiro $\theta_1 = 1.2$. O prior regulariza automaticamente.

Por que isso é importante para Data Science?

A Probabilistic View explica POR QUE usamos MSE (ruído Gaussiano) e POR QUE regularizamos (prior sobre parâmetros). Isso permite escolher a loss function corretamente para cada problema: ruído Gaussiano $\Rightarrow$ MSE, labels binários $\Rightarrow$ cross-entropy (ruído Bernoulli), dados de contagem $\Rightarrow$ Poisson deviance.

Aplicação Prática em Machine Learning

Em sklearn: Ridge e Lasso implementam MAP com Gaussian e Laplace prior respectivamente. Em PyTorch: o parametro `weight_decay` no Adam implementa exatamente $-\log p(\theta)$ para um prior Gaussiano. Para definir priors customizados, use Pyro ou PyMC (frameworks de probabilistic programming).

2. Prior Choices — Gaussian => L2 e Laplace => L1

A escolha do **Prior $p(\theta)$** determina qual tipo de regularização surge na otimização. Dois priors são especialmente importantes por terem formas fechadas e interpretação geométrica clara.

Gaussian Prior => L2 Regularization (Ridge / Weight Decay):

$$p(\theta) = \exp(-\|\theta\|^2) \Rightarrow -\log p(\theta) = \|\theta\|^2 \Rightarrow \min_{\theta} \text{MSE} + \lambda \|\theta\|^2$$

Gaussian: caudas leves, penaliza pesos grandes suavemente — encolhimento uniforme

Laplace Prior => L1 Regularization (Lasso / Basis Pursuit):

$$p(\theta) = \exp(-\|\theta\|_1) \Rightarrow -\log p(\theta) = \|\theta\|_1 \Rightarrow \min_{\theta} \text{MSE} + \lambda \|\theta\|_1$$

Laplace: cuspide em zero — acredita fortemente que $\theta = 0$ => esparsidade!

- **Use L2 (Gaussian Prior)** quando: acredita que todos os parâmetros são relevantes mas devem ser pequenos. Numericamente estável, tem solução analítica $(X^T X + \lambda I)^{-1} X^T y$.
- **Use L1 (Laplace Prior)** quando: acredita que muitos parâmetros são irrelevantes (deseja **feature selection** automática). Produz soluções esparsas — muitos $\theta_j = 0$ exatamente.
- **Diferença geométrica:** Gaussiana tem caudas leves (penaliza pesos grandes suavemente). Laplace tem caudas pesadas e uma cuspide em zero — a probabilidade máxima está em zero!
- **Ambos os priors centrados em zero:** expressam a crença de que parâmetros menores são mais prováveis a priori — equivale a dizer que o modelo não deve ser desnecessariamente complexo.

Bayesian Connection: Prior => Regularização

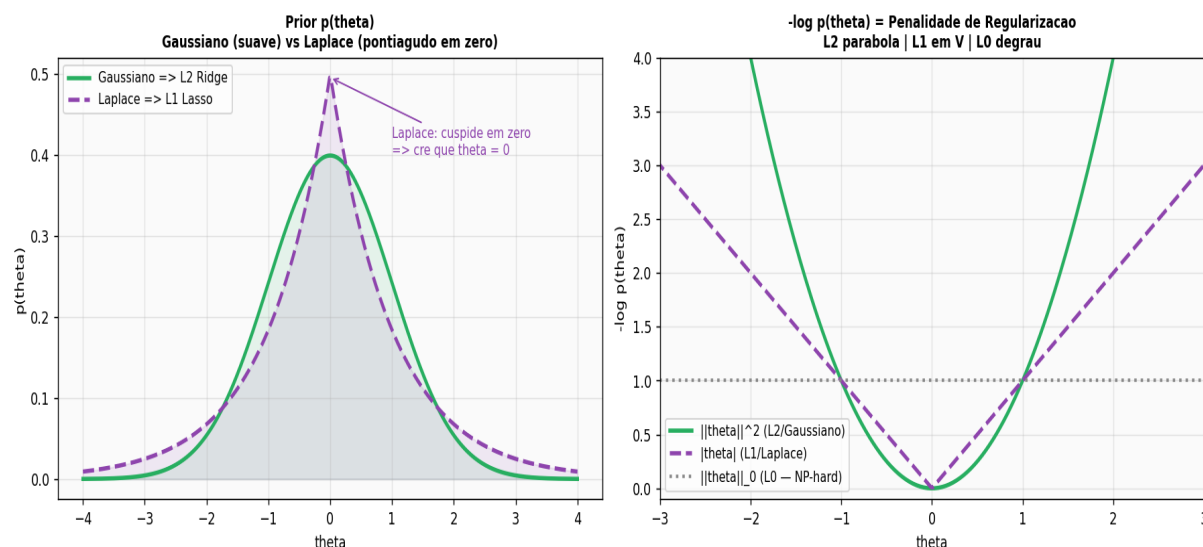


Figura 2 — Esquerda: Prior $p(\theta)$. A Gaussiana é suave e simétrica; a Laplace tem um pico pontagudo em zero, expressando crença forte de que θ deveria ser zero. Direita: $-\log p(\theta)$ = penalidade de regularização. L2 (parábola), L1 (em V com

cuspside em 0), L0 (degrau — NP-hard). A cuspside do L1 em zero e a razao geometrica da esparsidade.

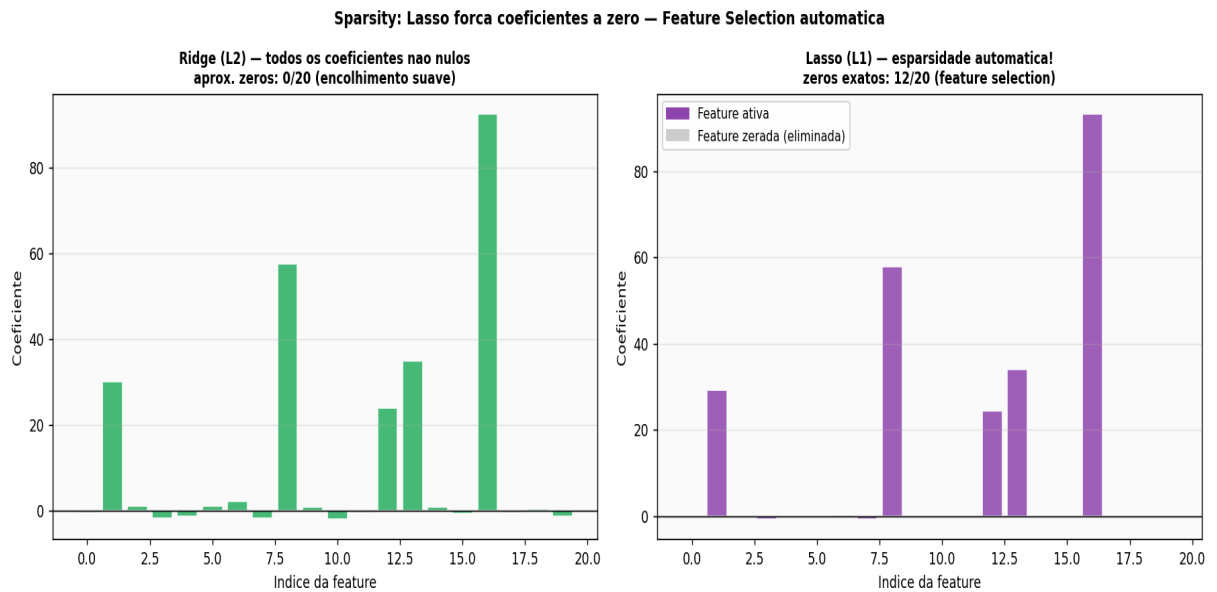


Figura 3 — Ridge (L2) vs Lasso (L1) nos mesmos dados com 20 features e apenas 5 informativas. Ridge encolhe todos os coeficientes mas nenhum chega a zero exatamente (barras verdes). Lasso zera automaticamente as features irrelevantes (barras cinzas) — fazendo Feature Selection.

Por que isso e importante para Data Science?

A dualidade Gaussian/L2 vs Laplace/L1 e o framework unificado de regularizacao. Conhecendo a conexao Bayesiana, voce pode definir seus proprios priors para problemas especificos: prior de simetria, prior de suavidade, prior de positividade. Cada prior se traduz em um regularizador diferente com propriedades especificas.

Aplicacao Pratica em Machine Learning

sklearn: Ridge(alpha=lambda) para L2, Lasso(alpha=lambda, max_iter=10000) para L1. Lembrete: sempre normalize as features com StandardScaler antes de aplicar Ridge/Lasso — a regularizacao e sensivel a escala! Sem normalizacao, features com escala maior sao penalizadas de forma desproporcional.

3. Decision Process — Quando Usar Qual Regularizacao?

Nao existe um metodo universal. O slide apresenta um **processo de decisao** estruturado em 4 cenarios baseados nas caracteristicas do problema. O ponto de partida e sempre a estrutura dos dados e do problema.

Cenario	Recomendacao	Dicas Praticas
Small data + many features (N pequeno, p grande)	L2 Ridge ou L1 Lasso Early Stopping Arquitetura simples	Standardizacao + CV para lambda Comece com Ridge, tente Lasso para feature selection
Structured Outputs (imagens, series temporais, grafos)	TV / Laplace (Output Regularization) Variantes estruturais	TV (Total Variation) preserva bordas Smoothing priors para series temporais
Strong Prior Knowledge (dominio especifico)	Regularization by Design Constraints fisicas	Limitar capacidade pela estrutura Usars inductive bias do dominio
Optimization Unstable (gradientes explodem/nan)	Gradient Clipping Max-Norm por camada Nao-negatividade	Gradient clip por norma (max_norm=1.0) Reduzir Learning Rate

Hierarquia de decisao recomendada:

- **Passo 1 — Volume de dados:** $N < 1000 \Rightarrow$ sempre regularizar (L2 ou L1). $N > 100k \Rightarrow$ regularizacao menos critica, foco em arquitetura.
- **Passo 2 — Numero de features:** $p \gg N \Rightarrow$ L2 obrigatorio (OLS singular). $p \ll N$ com esparsidade esperada $\Rightarrow$ L1 ou Elastic Net.
- **Passo 3 — Estrutura do output:** saidas com estrutura espacial/temporal $\Rightarrow$ Output Regularization (TV, Laplacian smoothing).
- **Passo 4 — Estabilidade do treino:** gradientes explodem $\Rightarrow$ Gradient Clipping e/ou Max-Norm. Pesos crescem sem limite $\Rightarrow$ Weight Decay (L2 implicito).
- **Passo 5 — Interpretabilidade:** precisa saber quais features importam $\Rightarrow$ Lasso ou Elastic Net para sparsidade automatica.

Por que isso e importante para Data Science?

O Decision Process evita o erro de aplicar a mesma regularizacao em todos os problemas. Ridge e excelente para regressao com todas as features relevantes. Lasso e melhor quando se suspeita que a maioria das features e irrelevante. Output Regularization e frequentemente ignorada mas essencial para problemas com saidas estruturadas (imagens, sinais, grafos).

Aplicacao Pratica em Machine Learning

Cenario mais comum na pratica (Small data + many features): StandardScaler() + Ridge(alpha=lambda) com lambda via RidgeCV ou GridSearchCV. Para Deep Learning com gradientes instaveis: torch.nn.utils.clip_grad_norm_(params, max_norm=1.0). Para Max-Norm: implementar manualmente com torch.renorm() apos cada passo de otimizacao.

4. Hyperparameter Choice — Guidelines e Protocolo

Apos escolher o tipo de regularizaco, e preciso encontrar o valor otimo de lambda **sem contaminar o conjunto de teste**. O slide apresenta um protocolo estruturado em 3 eixos: normalizaco, busca e estabilidade.

Ranges tipicos para cada hiperparametro:

Hiperparametro	Faixa Tipica	Observaco
L2 (Ridge / Weight Decay)	1e-6 ... 1e-2	Sempre Log-Grid; valores menores para N grande
L1 (Lasso)	1e-6 ... 1e-3	Lasso mais sensivel — faixa estreita
Max-Norm (por camada)	1 ... 3	Limita a norma maxima dos pesos por camada de rede neural
Dropout rate	0.1 ... 0.5	0.5 para MLP denso, 0.1-0.2 para conv. e transformer
Patience (Early Stopping)	5 ... 10 epocas	Monitorar val loss; restaurar melhor checkpoint

Por que Log-Grid e nao Linear-Grid?

$$\lambda \in \{10^{-6}, 10^{-5}, \dots, 10^{-1}, 10^0, 10^1, \dots, 10^2\}$$

np.logspace(-6, 2, 30) — lambda age multiplicativamente, nao aditivamente

Protocolo completo de escolha de hiperparametros (6 passos):

- **Passo 1 — Split limpo:** Train / Val / Test sem contaminaco. Estratificaco para classificaco. Nunca olhar o Test antes do fim.
- **Passo 2 — Coarse Sweep:** cada regularizador com lambda em Log-Grid amplo (`np.logspace(-6, 2, 30)`). Poucas epocas para eficiencia.
- **Passo 3 — Refinamento:** apenas nas top 2-3 regioes promissoras. Log-Grid estreito ao redor do melhor lambda coarse.
- **Passo 4 — Multi-seed:** media sobre 3-5 seeds se N for pequeno ($N < 500$). Reduz variabilidade da estimativa de performance.
- **Passo 5 — Modelo final:** retreinar em Train+Val com o melhor lambda. Mais dados de treino = melhor estimativa dos parametros.
- **Passo 6 — Teste UMA VEZ:** reportar no conjunto de teste apenas uma vez. Nunca usar o Test para tomar decisoes — isso vazaria informaco.

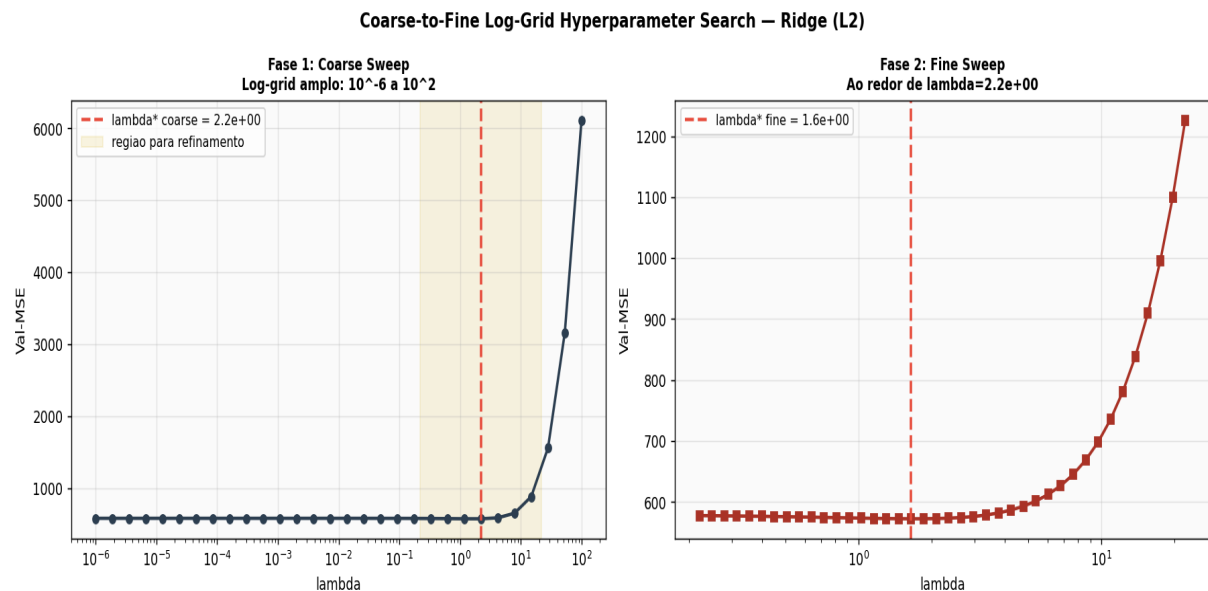


Figura 4 — Coarse-to-Fine Log-Grid Search. Fase 1 (esquerda): sweep amplo de 10^{-6} a 10^2 identifica a região ótima (zona amarela). Fase 2 (direita): sweep fino ao redor do melhor λ coarse refina a estimativa com custo computacional mínimo. A escala logarítmica é essencial para ver o detalhe onde importa.

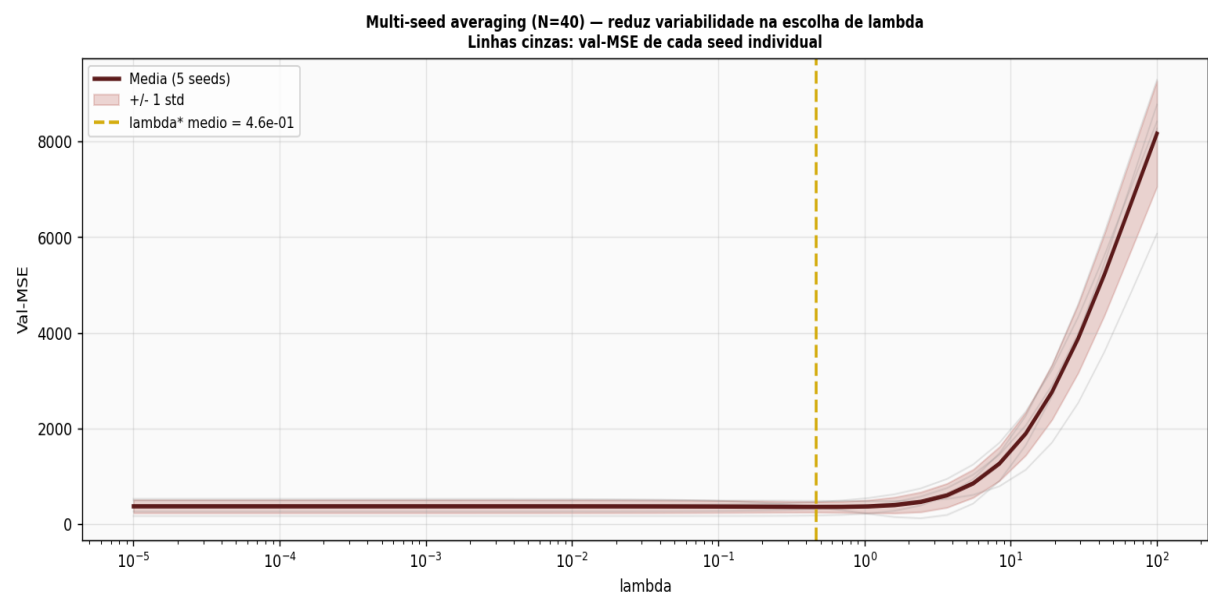


Figura 5 — Multi-seed averaging com $N=40$ (dataset pequeno). Linhas cinzas: val-MSE de cada seed individual — alta variabilidade. Linha escura: média dos 5 seeds com banda de ± 1 std. A média é muito mais estável e confiável para escolher λ em N pequeno.

Por que isso é importante para Data Science?

O protocolo de 6 passos previne os dois erros mais comuns em ML: (1) data leakage — contaminar o Test durante a escolha de hiperparâmetros; (2) lambda tuning em escala linear — desperdiçando a maioria dos pontos avaliados. Seguir o protocolo e a diferença entre uma avaliação credível e uma ilusão estatística.

Aplicação Prática em Machine Learning

sklearn: `RidgeCV(alphas=np.logspace(-6,2,30), cv=5).fit(X_tr,y_tr)` — implementação eficiente que faz CV internamente. LassoCV idem para L1. Para Deep Learning: ModelCheckpoint + EarlyStopping no PyTorch Lightning. Para multi-seed: usar `cross_val_score` com `random_state` diferente em cada call.

5. Diagnosis — Under vs. Over-Regularization

Apos escolher lambda, e essencial **diagnosticar** se a regularizacao esta corretamente calibrada. O instrumento padrao e a **Learning Curve**: plotar o erro de treino e validacao em funcao do tamanho do dataset N.

$$\text{gap}(N) = \text{Val-MSE}(N) - \text{Train-MSE}(N)$$

gap grande => High Variance (Under-Reg.) | gap pequeno mas alto => High Bias (Over-Reg.)

Padrao da Learning Curve	Diagnostico	Acao Recomendada
Erro treino BAIXO, erro val. ALTO, GAP GRANDE	Under-Regularizacao — High Variance / Overfitting	Aumentar lambda; Early Stopping mais agressivo; mais dados
Erro treino ALTO, erro val. ALTO, GAP PEQUENO	Over-Regularizacao — High Bias / Underfitting	Diminuir lambda; modelo mais complexo; mais features
Erro treino aprox. erro val. aprox. nivel aceitavel	Regularizacao otima — bom equilibrio	Manter lambda; investir em mais dados ou novas features

Diagnosis via Learning Curves — Under / Correct / Over-Regularization

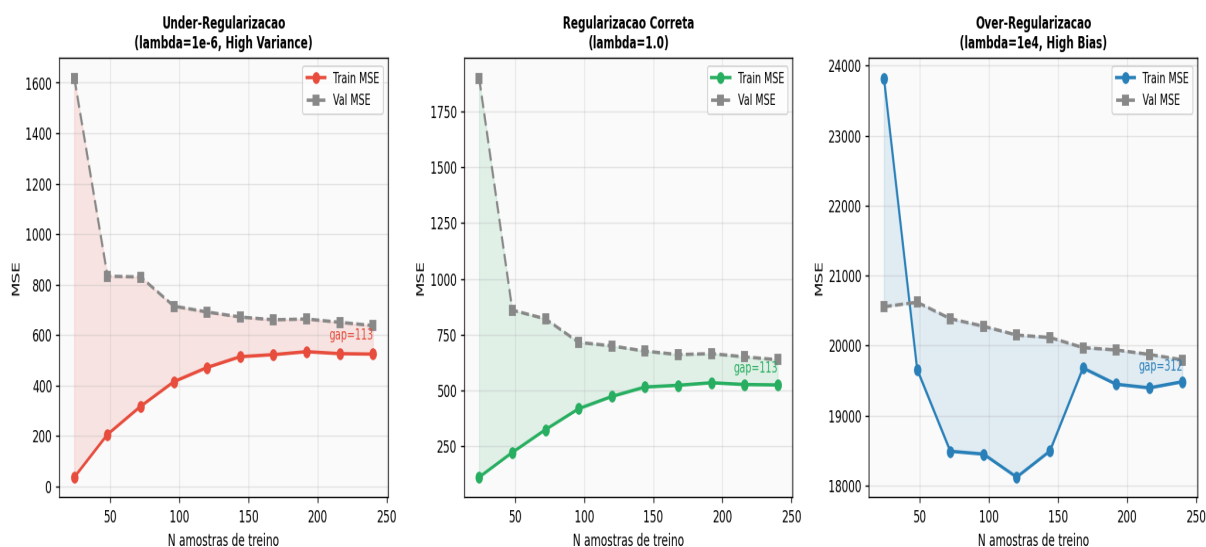


Figura 6 — Learning Curves para tres valores de lambda. Esquerda (Under-Regularizacao, lambda=1e-6): gap enorme entre treino e validacao. Centro (lambda=1.0): gap fecha conforme N cresce — bom calibracao. Direita (Over-Regularizacao, lambda=1e4): ambos os erros altos e proximos — modelo nao aprende nada util.

Por que isso e importante para Data Science?

O diagnostico via Learning Curves e o mapa de orientacao para qualquer problema de regularizacao. Sem ele, qualquer ajuste de lambda e cego. Gap grande indica High Variance — a solucao e mais regularizacao OU mais dados. Ambos erros altos indica High Bias — a solucao e menos regularizacao OU modelo mais rico.

Aplicacao Pratica em Machine Learning

sklearn: `learning_curve(modelo, X, y, train_sizes=np.linspace(0.1,1,10), cv=5, scoring='neg_mean_squared_error')`. Plotar train vs val MSE vs N. Para Deep Learning: monitorar train loss e val loss por epoca no tensorboard ou wandb. O padrao e identico — apenas o eixo x muda de N para epocas.

6. Cross-Validation — Estimativa Confiavel do Erro Real

Cross-Validation e uma familia de tecnicas para estimar como um modelo performa em dados nao vistos. Pode ser usada para avaliar modelos, selecionar entre candidatos, ajustar hiperparametros e evitar overfitting ao conjunto de validacao. **Principio:** nunca use o conjunto de teste para tomar decisoes.

Estimativa K-Fold (formula central):

$$\hat{Err}_{K-Fold} = \frac{1}{K} \sum_{k=1}^K \mathcal{L}(f_{-k}, D_k)$$

f_{-k} : modelo treinado em todos os folds exceto o k-esimo | D_k : fold de validacao k

Variante	Mecanismo	Vantagem	Quando usar
Hold-Out	Split unico (ex: 75/25). Um treino, uma avaliacao.	Muito rapido — custo $O(1)$.	N muito grande (>50k)
K-Fold (K=5 ou 10)	K splits rotacionados; media dos K erros.	Balanea custo e confiabilidade.	Padrao para a maioria dos casos
Stratified K-Fold	K-Fold preservando % de classes em cada fold.	Robusto a desbalanceamento de classes.	Classificacao com classes raras
Leave-One-Out (LOO)	K=N: cada amostra e validacao uma vez.	Minimo vies — usa N-1 amostras no treino.	N muito pequeno (<50-100)

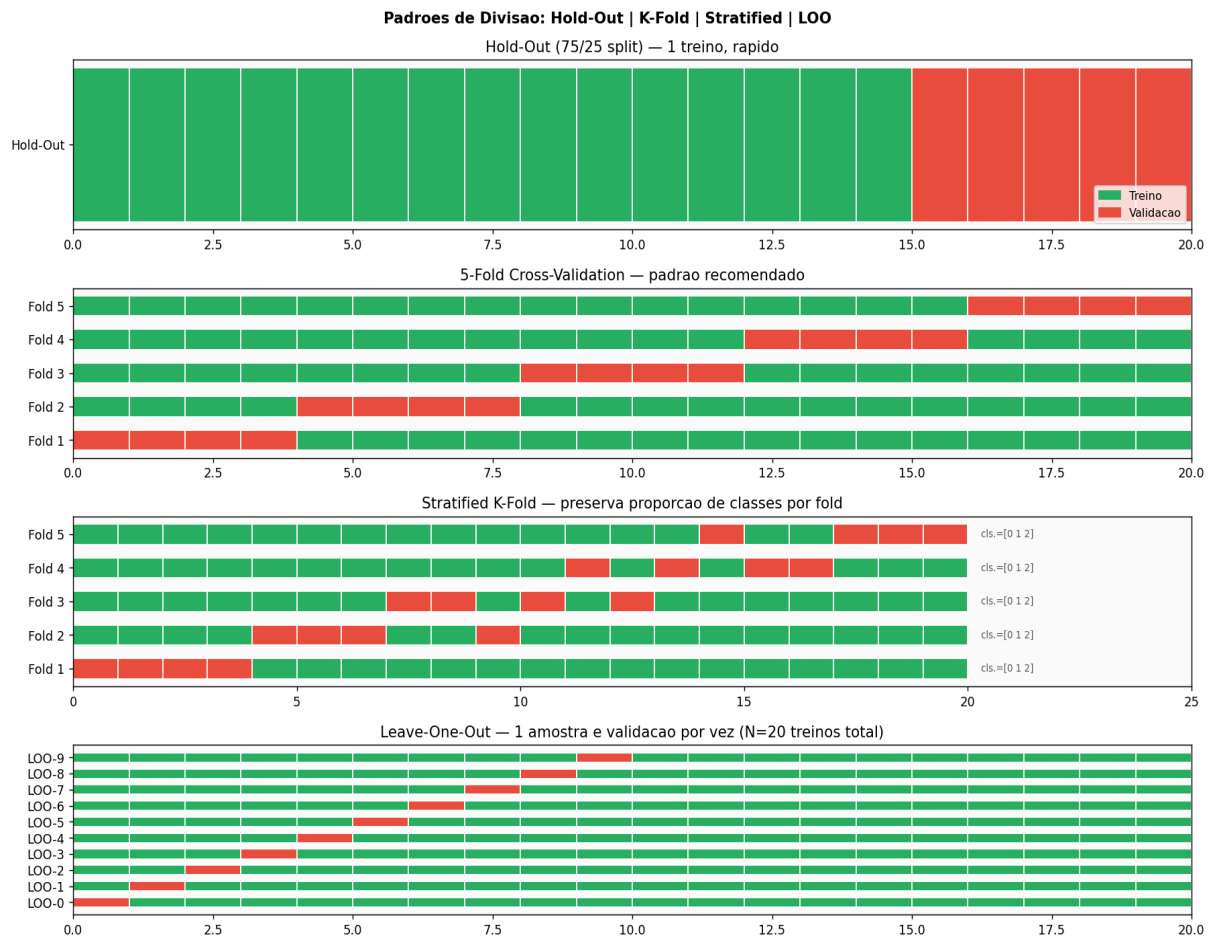


Figura 7 — Padroes de divisao: verde = treino, vermelho = validacao. Hold-Out: um unico split. K-Fold: cada fold e validacao uma vez de forma rotacionada. Stratified K-Fold: mesma logica, mas preserva a proporcao de classes em cada fold (essencial para classificacao desbalanceada). LOO: cada amostra e validacao uma vez.

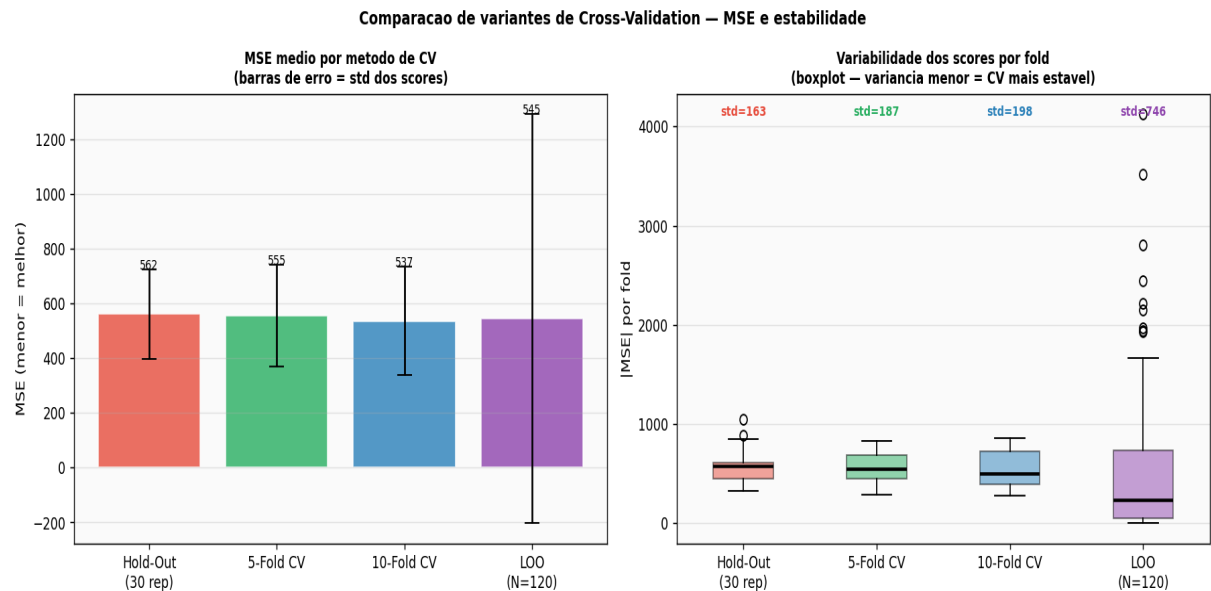


Figura 8 — Comparacao quantitativa: MSE medio e variabilidade por metodo. Hold-Out tem a maior variabilidade entre seeds. K-Fold (K=5 e K=10) oferecem estimativas mais estaveis. LOO e deterministico mas muito lento para N grande. O std indicado em cada barra e a variabilidade da estimativa.

Por que isso e importante para Data Science?

A variabilidade do Hold-Out (depende fortemente do split aleatorio) e o motivo pelo qual K-Fold e o padrao. Com K=5 e shuffle=True, a estimativa e robusta e computacionalmente razoavel. Stratified K-Fold e obrigatorio em qualquer problema de classificacao com desbalanceamento — Hold-Out padrao pode ter 0% de positivos em algum fold, tornando a estimativa completamente errada.

Aplicacao Pratica em Machine Learning

sklearn: KFold(n_splits=5, shuffle=True, random_state=42) + cross_val_score(modelo, X, y, cv=kf, scoring='neg_mean_squared_error'). Para classificacao: StratifiedKFold. Para series temporais: TimeSeriesSplit (nunca use KFold com dados temporais — vazaria o futuro para o treino!)

7. Bootstrapping — Resampling, Incerteza e Bagging

Bootstrapping é uma técnica de **resampling** para quantificar a incerteza de um estimador sem precisar de formulas analíticas. **Ideia central:** o dataset observado D é tratado como uma **população empírica**. Reamostrar D **com reposição** simula o processo de coletar novos datasets da população real. A variabilidade entre reamostras aproxima a incerteza do estimador.

Algoritmo Nonparametric Bootstrap:

- **Input:** dados $D = \{z_1, \dots, z_N\}$, estatística $S(D)$, replicatas B (tipicamente 500-2000).
- **Para $b = 1, \dots, B$:** amostrar D^b com reposição (N amostras); calcular $s^b = S(D^b)$.
- **Output:** distribuição $\{s^1, \dots, s^B\}$ — aproxima a distribuição amostral de S .
- **Erro padrão estimado:** $\text{std}(\{s^b\})$. **IC 95%:** percentis 2.5% e 97.5% da distribuição bootstrap.
- **Importante:** o mesmo pipeline completo deve rodar dentro de S — preprocessing, feature selection, tuning — para evitar vazamento de informação.

Tres tipos de Bootstrap Confidence Intervals (95%):

Tipo	Formula / Logica	Quando usar
Normal approximation	$\hat{s} \pm 1.96 * \text{std}(\{s^b\})$	Distribuição bootstrap aproximadamente Normal
Percentile interval	$[q_{2.5\%}(\{s^b\}), q_{97.5\%}(\{s^b\})]$	Distribuição assimétrica — mais robusto
BCa (Bias-Corrected Accelerated)	Corrige vies e aceleração — mais complexo	Distribuição enviesada — mais preciso

Bootstrap Coefficient Uncertainty (B=500) — (*) IC não inclui zero => feature significativa

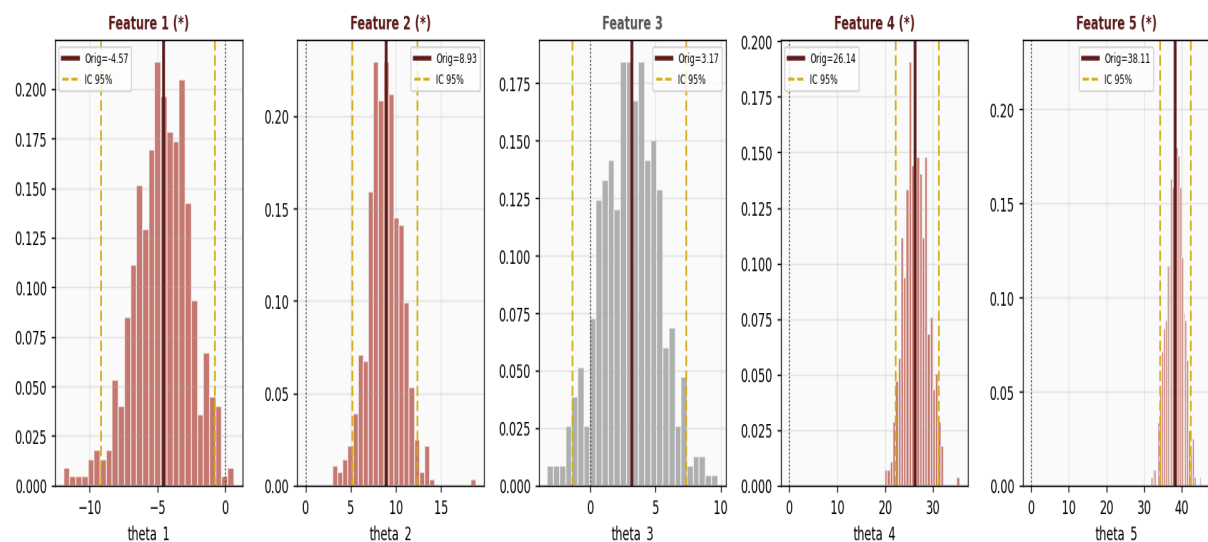


Figura 9 — Distribuição bootstrap dos coeficientes ($B=500$ replicatas). Histograma: distribuição de cada coeficiente sobre as replicatas. Linha escura: estimativa original no dataset completo. Linhas amarelas: IC 95% percentile. Features marcadas com (*) tem IC que não inclui zero — provavelmente significativas.

Bagging — Bootstrap Aggregating:

Bagging é o uso de Bootstrap para reduzir variância: treinar o mesmo modelo em B amostras bootstrap diferentes e calcular a média das previsões. É a base do **Random Forest** (Bagging de árvores de decisão). A média de B estimadores instáveis produz um estimador muito mais estável.

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$$

Cada $\hat{f}^{(b)}$ é treinado em uma amostra bootstrap D^b — alta variância individual, baixa na média

Bagging = Bootstrap + Averaging => reduz variância

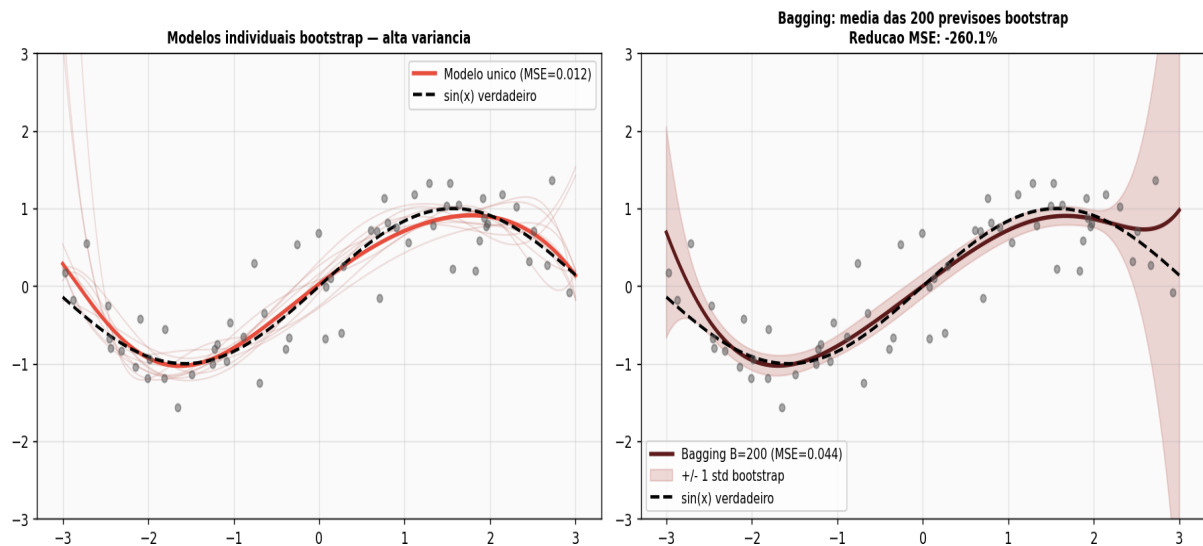


Figura 10 — Bagging com polinômio de grau 7. Esquerda: modelos individuais ($B=200$ linhas finas vermelhas) — alta variância, muita oscilação. Direita: média das B previsões bootstrap (linha grossa escura) e muito mais próxima de $\sin(x)$ verdadeiro, com banda de incerteza coerente. A redução de MSE em relação ao modelo único demonstra o poder do Bagging.

Por que isso é importante para Data Science?

Bootstrapping é a base teórica do Random Forest (Bagging de árvores). Além de reduzir variância, o Bootstrap também fornece as amostras Out-of-Bag (OOB) — as amostras não incluídas em cada replicata — que podem ser usadas como um conjunto de validação gratuito, sem precisar de CV separado.

Aplicação Prática em Machine Learning

sklearn: `BaggingRegressor(Ridge(), n_estimators=200, bootstrap=True)`. Random Forest já implementa Bagging + seleção aleatória de features. Para IC de coeficientes: implementar Bootstrap manualmente com loop `'for b in range(B): idx = np.random.choice(N, N, replace=True)'`. Percentile IC: `np.percentile(coefs_boot, [2.5, 97.5], axis=0)`.

8. Resumo Final — Mapa Conceitual da Aula 4

Conceito	Ideia Central	Formula / Ferramenta Chave
Probabilistic View	MSE emerge do ruído Gaussiano; regularização e Prior Bayesiano	$-\log p(D \theta) = \text{MSE}$ $-\log p(\theta) = \text{Regularização}$
MAP Estimation	Minimizar Loss + Regularizador = Maximizar Posterior	$\theta_{\text{MAP}} = \text{argmin} [\text{MSE} + \lambda * R(\theta)]$
Gaussian Prior => L2	Prior Gaussiano => penalidade $\ \theta\ ^2$ => Ridge	<code>Ridge(alpha=lambda).fit(X, y)</code>
Laplace Prior => L1	Prior Laplace => penalidade $\ \theta\ _1$ => Lasso (esparsidade)	<code>Lasso(alpha=lambda, max_iter=10000).fit(X, y)</code>
Decision Process	4 cenários: Small data, Structured Output, Prior Known, Unstable Opt.	Hierarquia: volume -> features -> output -> estabilidade
Log-Grid Search	Lambda age multiplicativamente => busca em Log-Grid, não linear	<code>np.logspace(-6, 2, 30) + Coarse-to-Fine</code>
Diagnosis	Learning Curves diagnosticam High Variance vs High Bias	gap grande => Under-Reg. ambos altos => Over-Reg.
K-Fold CV	Estimativa confiável do erro real sem contaminar o Test	<code>KFold(5, shuffle=True) + cross_val_score(...)</code>
Bootstrap	Resampling com reposição => distribuição de qualquer estatística	IC Percentile: <code>np.percentile(s_boot, [2.5, 97.5])</code>
Bagging	Média de B modelos bootstrap => reduz variância drasticamente	<code>BaggingRegressor(base, n_estimators=200, bootstrap=True)</code>

Checklist para qualquer novo projeto:

- Escolher loss function baseada na distribuição do ruído (Gaussiano => MSE, Bernoulli => cross-entropy).
- Definir prior implícito: L2 para todos relevantes, L1 para feature selection, sem prior para MLE.
- Usar Log-Grid para busca de lambda — NUNCA escala linear.
- Aplicar Coarse-to-Fine para eficiência computacional.
- Diagnosticar via Learning Curves antes de aceitar qualquer lambda.
- Validar com K-Fold (K=5 ou 10), Stratified para classificação desbalanceada.
- Usar Bootstrap para quantificar incerteza de coeficientes ou métricas em N pequeno.
- Retreinar em Train+Val com o melhor lambda. Reportar no Test UMA ÚNICA VEZ.

Na próxima aula, exploraremos **metodos baseados em arvores de decisao** — Random Forests (Bagging de arvores visto nesta aula) e Gradient Boosting (Boosting como forma de redução de Bias). Os conceitos de Probabilistic View, Cross-Validation e Bootstrap desta aula são idênticos para avaliar e tunar esses modelos.